软件需求分析与设计

第八章框架梳理

仅供学习参考

目 录

一、 业务对象和持久性 - 1 -

（一） 数据库管理系统 - 1 -

1. 实体类和持久类 - 1 -

2. 数据库的分类 - 1 -

3. 数据模型（数据库模式）层次 - 1 -

4. 集成应用系统与数据库建模 - 1 -

5. 对象-数据库映射基础 - 1 -

二、 关系数据库模型 - 2 -

（一） 关系数据库（RDB） - 2 -

1. 原始类型 - 2 -

2. RDB 不支持的原始类型 - 2 -

3. RDB 模型 - 2 -

（二） 列、域和规则 - 2 -

1. 列（column） - 2 -

2. 域（domain） - 2 -

3. 业务规则（rules） - 2 -

（三） 表（table） - 3 -

1. 表的组成 - 3 -

（四） 参照完整性（Referential integrity） - 3 -

1. 参照完整性的作用 - 3 -

2. 外码 <fk> - 3 -

3. 更新操作 - 3 -

（五） 触发器（Trigger） - 3 -

1. 触发器的定义 - 3 -

（六） 存储过程（Stored Procedure） - 3 -

1. 存储过程的定义 - 3 -

（七） 视图（Views） - 3 -

1. 视图的定义 - 3 -

（八） 范式（Normal forms） - 4 -

仅供学习参考

1. 范式的分类 - 4 -
2. 不同范式间的关系 - 4 -
三、 对象—关系映射 - 5 -
（一） 映射的种类 - 5 -
1. 映射实体类 - 5 -
2. 映射关联 - 5 -
3. 映射聚合 - 5 -
4. 映射泛化 - 5 -
四、 管理持久对象的模式 - 6 -
（一） 企业应用体系结构模式（Patterns of Enterprise Application Architecture） - 6 -
1. 标识映射（Identity Map） - 6 -
2. 数据映射（Data Mapper） - 6 -
3. 延迟装载（Lazy Load） - 6 -
4. 工作单元（Unit of work） - 6 -
（二） 检索持久对象 - 6 -
（三） 装载（Load）持久对象 - 6 -
（四） 释放（Unload）持久对象 - 6 -
五、 管理持久对象的模式 - 7 -
（一） SQL 程序设计的层次 - 7 -
（二） 设计业务事务 - 7 -
1. 事物 - 7 -
2. 锁 - 7 -
3. 隔离层次： - 7 -
4. 自动恢复 - 7 -

仅供学习参考

一、业务对象和持久性

（一）数据库管理系统

1. 实体类和持久类

(1) 实体类表示应用程序中的持久数据库对象，但实体类**不是**数据库中的持久类

2. 数据库的分类

(1) 关系型（如 Sybase、DB2、Oracle）——最常用

(2) 对象—关系型（如 UniSQL、Oracle）

(3) 面向对象型（如 ObjectiveStore、Versant）

3. 数据模型（数据库模式）层次

(1) 数据模型的定义

① **数据模型**是用比原始的位和字节更易于理解的词来表示数据库结构的一种抽象

② 过去数据库主要把精力放在**数据模型**（UML 称为静态模型）上

③ 当今数据库存储和执行程序的功能已经扩展到**行为模型**（以触发器和存储过程为中心）

(2) 数据模型的 3 个层次

① 外部（概念）数据模型

1) **实体关系（ER）图**——最流行的概念数据建模技术

② 逻辑数据模型

1) 逻辑模式，也称为全局概念模式

2) 能够反映系统实现时所采用的数据库模型的逻辑存储结构（表等）的模型

③ 物理数据模型

1) 专门针对特定的 DBMS

2) 定义数据如何真正存储在持久存储设备上，通常指磁盘

4. 集成应用系统与数据库建模

(1) PCBMER 中的向下依赖原则（DDP）扩展了应用程序与数据库间的通信

(2) **面向对象数据库模型**可以用作客户端应用程序和服务器端数据源之间协同工作的对象存储 API

5. 对象-数据库映射基础

(1) 映射困难的原因：

① 数据库的存储结构处理不了面向对象泛型

② 数据库几乎都不是为了单个应用程序设计的

仅供学习参考

二、关系数据库模型

（一）关系数据库（RDB）

1. 原始类型

(1) 为建模和程序设计提供的固有数据类型称为原始类型

2. RDB 不支持的原始类型

(1) 对象类型及相关的概念（如继承和或方法）

(2) 结构化类型

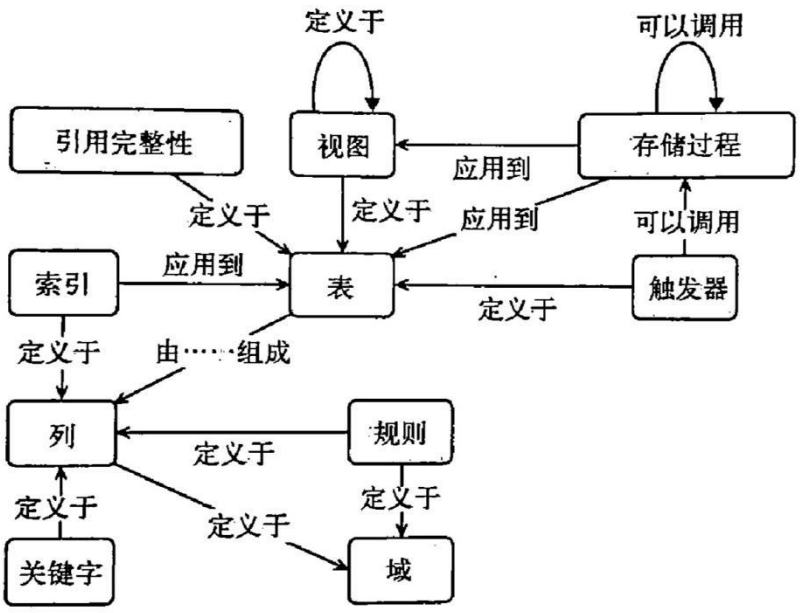
(3) 集合

(4) 引用

3. RDB 模型

(1) RDB 模型的主要原始类型是由多列组成的关系表

(2) 表的列只能取原子值——结构化值或值的集合是不允许的



数据库原始类型之间的依赖关系

（二）列、域和规则

1. 列（column）

(1) 关系型数据库采用由列和行组成的表来定义数据

(2) 具有单一定义域，具有原子域（数据类型）

① 原子域（atomic domains）：数据值必须是简单的（不可分割的）和单一的（不重复的）值

2. 域（domain）

(1) 定义了一个列可以取值的合法集（可以匿名）

3. 业务规则（rules）

(1) 默认值（default）

(2) 取值范围（range）

(3) 值列表（list）

(4) 值大小写

(5) 值格式（format）

（三）表（table）

1. 表的组成

(1) 关系表定义为固定列的集合，表中的列可以是固有类型或用户自定义类型（域）

(2) 表中的行数量没有限制，特定行中的列值允许为 null

(3) 表在数学上是一个集合，因此不能有重复的行（主码<pk>约束）

（四）参照完整性（Referential integrity）

1. 参照完整性的作用

(1) 通过参照完整性约束才维护表间关系——外码

(2) CPA 约束（Change Parent allowed）——可以将子（外）表中的记录重新指派到父表中另一个记录上

2. 外码 <fk>

(1) 外码是指表中的一组列，它的值要么是 null，要么必须与同一个或另一个表中的主码值匹配

3. 更新操作

(1) Restrict——限制更新或删除操作

(2) Cascade——级联操作

(3) Set Null——设置为空值

(4) Default——设置为缺省值

（五）触发器（Trigger）

1. 触发器的定义

(1) 一个小程序，当定义了触发器的表发生修改操作时（如 insert，update，delete）自动执行（触发）

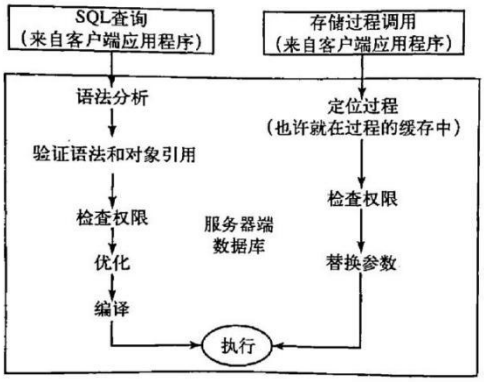
(2) SQL 语句格式：create trigger <NAME> on <TABLE> for <OPERATION> as <CODE>

（六）存储过程（Stored Procedure）

1. 存储过程的定义

(1) 必须指定名字，可以有输入、输出参数

(2) 执行效率比传统查询高，性能的优势在于缓解了网络堵塞



客户端SQL与存储过程调用的比较

（七）视图（Views）

1. 视图的定义

(1) 视图的本质是一条 select 语句，把这条语句起了个名

(2) 关系视图是被存储和命名的 SQL 查询

(3) 视图并不存储持久数据，只是将数据的快照展现出来

（八）范式（Normal forms）

1. 范式的分类

   (1) 第一范式

   (2) 第二范式

   (3) 第三范式

   (4) BCNF 范式

   (5) 第四范式

   (6) 第五范式

2. 不同范式间的关系

   (1) 高范式的表能满足所有低范式

   (2) 低范式的表可能会出现**更新异常**——对表进行修改操作所引起的意外的连带后果（数据不一致）

仅供学习参考

三、对象—关系映射

（一）映射的种类

1. 映射实体类

   (1) 实体类到关系表的映射必须满足表的第一范式，列必须是原子的

   (2) 如果类图中涉及集合类型，映射时将集合字段属性拆开单独成表 Web 应用程序

2. 映射关联

   (1) 一对一关联——外码选取任意一方都可

   (2) 一对多关联 / 多对一关联——外码选取“多”的一方

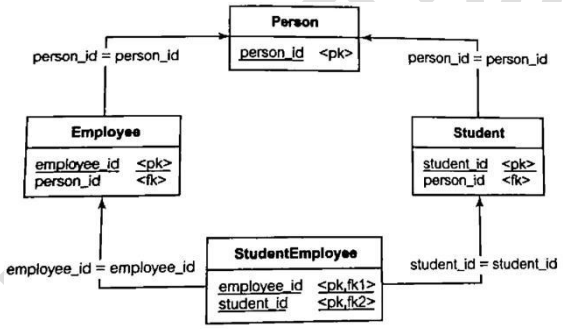
   (3) 多对多关联——需要额外一个交叉表

3. 映射聚合

   (1) 具有特殊语义的一对多的关联

4. 映射泛化

   (1) 第一种映射策略——每个类映射到一个表——存在数据冗余、信息不一致问题



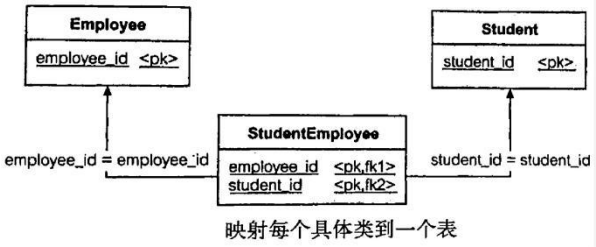
每个类映射到· -个表

   (2) 第二种映射策略——类层次映射到一个表——存在数据冗余、信息不一致问题

Person			
person_id	uniqueidentifier	<pk>	not null
is_employee	char(1)		null
is_student	char(1)		null

类层次映射到一个表

   (3) 第三种映射策略——映射每个具体类到一个表——存在数据冗余、信息不一致问题



映射每个具体类到一个表

   (4) 第四种映射策略——映射每个没有连接的具体类到一个表——存在数据冗余、信息不一致问题

Employee			
employee_id	NCHAR(8)	<pk>	not null
is_student	BOOLEAN		not null

Student			
student_id	NCHAR(10)	<pk>	not null
is_employee	BOOLEAN		not null

映射每个没有连接的具体类到一个表

四、管理持久对象的模式

（一）企业应用体系结构模式（Patterns of Enterprise Application Architecture）

1. 标识映射（Identity Map）

(1) 给内存中所有持久对象都指定对象标识符（OID），没在内存中的对象不管

2. 数据映射（Data Mapper）

(1) 程序随时都知道所需要的对象是在内存中还是从数据库中获得，具有当前内存中的对象的相关信息

(2) 如果对象在内存中，在标识映射表去找，如果不在则去加载进内存

3. 延迟装载（Lazy Load）

(1) 一个对象并没有包含你需要的所有数据，但知道如何获取

(2) 应用程序可以只装载一部分对象到内存中

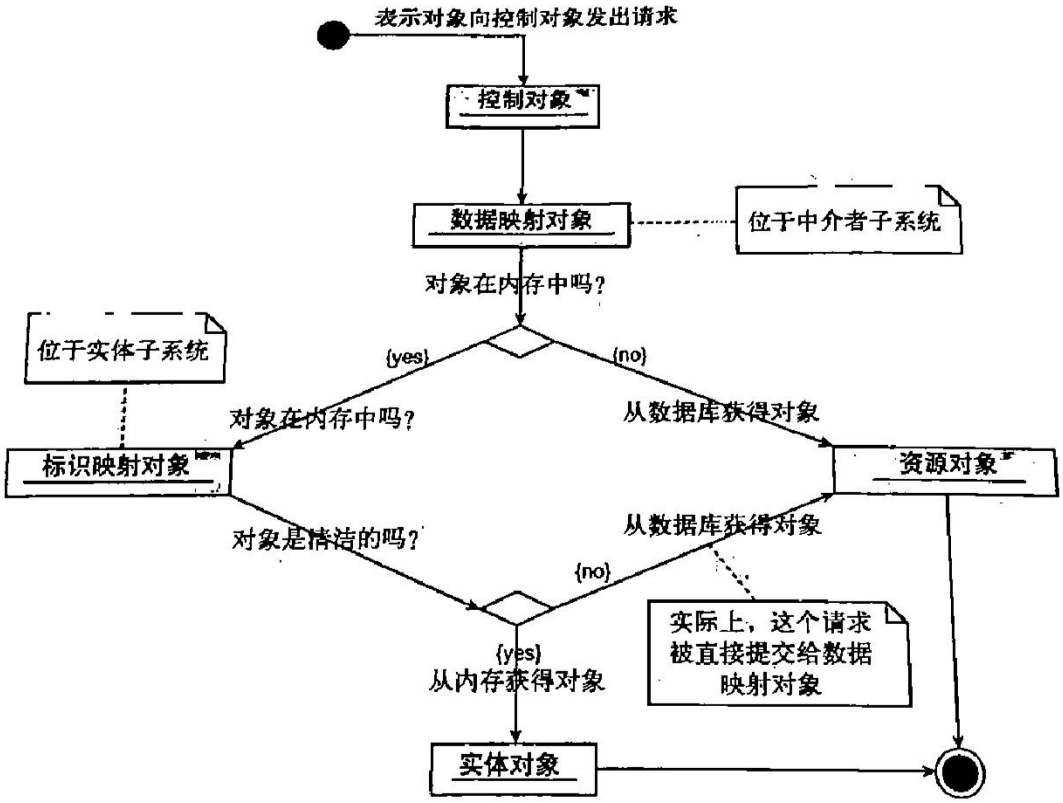
4. 工作单元（Unit of work）

(1) 程序知道一个业务事务都包含内存中的哪些对象

① 负责协调改变的写入和解决并发问题

② 和事务配套，随时可以回滚

（二）检索持久对象



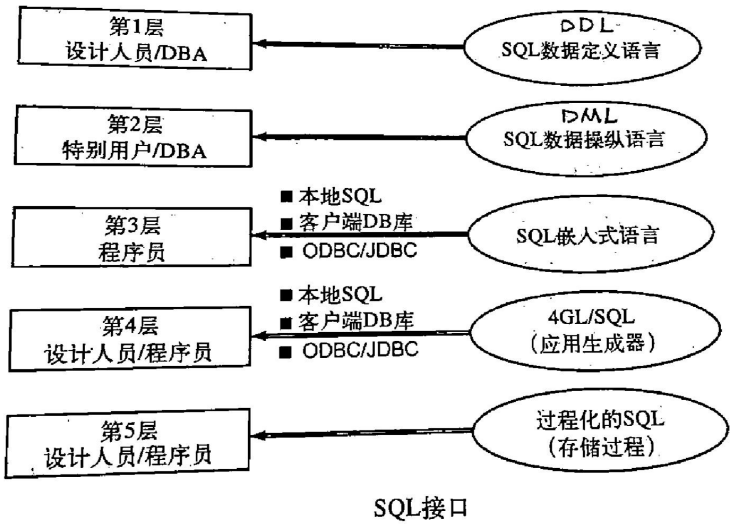
检索持久对象

（三）装载（Load）持久对象

（四）释放（Unload）持久对象

五、管理持久对象的模式

（一）SQL 程序设计的层次



（二）设计业务事务

1. 事物

   (1) 事务是一个逻辑工作单元，保证数据库的一致性

   (2) DBMS 事务管理有两个作用——数据库恢复和并发控制

   (3) 事务是原子的，事务中的所有 SQL 语句的结果要么被提交（commit），要么被回滚（rollback）

   (4) 长事务不允许自动回滚，只允许人工回滚

2. 锁

   (1) 排他（写）锁——必须等待当前持有锁的事务释放该锁之后才能处理其他事务

   (2) 更新（预写）锁——其他事务可以读取对象，但有需要可以随时升级为排他锁

   (3) 读（共享）锁——其他事务可以读取对象并可能得到这个对象的更新锁

   (4) 无锁——其他事务可以随时更新对象，因此它只适用于允许脏读的应用系统

3. 隔离层次：

   (1) 脏读（Dirty read possible）

   (2) 非重复读（Nonerepeatable read possible）

   (3) 虚读（Phantom possible）

   (4) 可重复读（保证事务的可序列化的执行）

4. 自动恢复

   (1) 如果事务没被提交，恢复可以回滚

   (2) 如果事务被提交后，恢复需要做一个逆操作

   (3) 可编程的恢复：

      ① 保存点（save point）——程序中的一条语句，把一个长事务分成若干个短事务，人为设置

      ② 触发器回滚，是一种特殊的保存点，允许用触发器回滚事务